Graph Analysis via Graph Rewriting: Sparse Linear Algebra Based Graph Analysis in Interaction Nets

Danil Zaripov[1,*], Efim Kubyshkin[1], Nikolai Ponomarev[1], Georgii Sichkar[1], Vladimir Zaikin[1] and Semyon Grigorev[1]

[1]*Saint Petersburg State University, 7-9 Universitetskaya Embankment, St. Petersburg, 199034, Russia*

Abstract

A clear and well-documented LaTeX document is presented as an article formatted for publication by CEUR-WS in a conference proceedings. Based on the "ceurart" document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.

Keywords

Interaction Nets, graph rewriting, graph analysis algorithms, GraphBLAS, sparse linear algebra, quad-tree

1. Introduction

Graph analysis has become fundamental to modern data science, powering applications ranging from recommendation systems in social networks [?] and fraud detection in financial transactions [?] to protein interaction analysis in bioinformatics [?] and static code analysis [?]. As graph sizes grow to billions of vertices and edges, the need for efficient, scalable implementations becomes critical. However, graph analysis algorithms are difficult to parallelize due to their irregular memory access patterns, poor data locality, and load imbalance across computational units [?].

The GraphBLAS standard [?] addresses these challenges by providing a foundation for constructing high-performance parallel graph analysis algorithms. By expressing graph algorithms in the language of linear algebra—using sparse vectors and matrices as core data structures, together with a rich set of highly parameterizable operations (matrix multiplication, element-wise operations, etc.)—GraphBLAS offers a high-level, mathematically rigorous framework for algorithm construction. Moreover, these linear algebraic primitives can be optimized for diverse hardware architectures, providing a path to high-performance graph analysis across platforms. This approach has proven successful, with implementations like SuiteSparse:GraphBLAS [?] delivering high performance on shared-memory systems, and LAGraph [?] providing a collection of graph analysis algorithms built on these primitives. However, designing parallel functions over sparse data structures remains a nontrivial engineering challenge, particularly for accelerators. Beyond CPU implementations, GraphBLAS has been partially ported to such devices: GraphBLAST [?] and Spla [?] target GPGPUs, while GraphLily [?] targets FPGAs.

Despite its successes, GraphBLAS and its implementations face certain challenges. The algebraic structures central to GraphBLAS—such as semirings and monoids—align more naturally with functional programming paradigms than with imperative ones. Generic data structures, such as matrices and vectors, and functions that required to provide a flexible core for matrix processing, together with user-defined types and operations (which are themselves functions), complicate API design and usage, particularly with respect to type safety. This insight suggests that functional programming languages could simplify the design of GraphBLAS-like APIs, offering cleaner abstractions and more

Woodstock'22: Symposium on the irreproducible science, June 07–11, 2022, Woodstock, NY

*Corresponding author.

st129060@student.spbu.ru (D. Zaripov); e.kubyshkin@spbu.ru (E. Kubyshkin); n.ponomarev@spbu.ru (N. Ponomarev); Manfred.Jeusfeld@acm.org (G. Sichkar); Manfred.Jeusfeld@acm.org (V. Zaikin); s.v.grigoriev@mail.spbu.ru (S. Grigorev)

0009-0003-1086-9279 (D. Zaripov); 0009-0009-5531-1758 (E. Kubyshkin); 0009-0000-2382-5687 (N. Ponomarev); 0000-0002-9421-8566 (G. Sichkar); 0000-0002-9421-8566 (V. Zaikin); 0000-0002-7966-0698 (S. Grigorev)

© 2022 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

composable primitives. However, general-purpose functional languages are often not fast enough for high-performance numerical computing. Research into functional languages and frameworks for parallel computations (including GPUs)—such as Futhark [?], AnyDSL [?], LIFT [?]—while demonstrating promising results, has largely focused on dense computations, leaving sparse irregular workloads less explored. Special hardware for graph analysis. "A Survey on Graph Processing Accelerators: Challenges and Opportunities", "Graph Processing on FPGAs: Taxonomy, Survey, Challenges"

At the same time, interaction nets [?]—a graph-rewriting model introduced by Yves Lafont—offer a computational model that is inherently well-suited for irregular parallelism. In interaction nets, computation proceeds through local rewriting rules applied to pairs of connected nodes (so-called active pairs). Crucially, multiple non-interfering active pairs can be reduced simultaneously, making interaction nets a promising foundation for parallel irregular computations, including graph analysis. Furthermore, interaction nets are relatively close to functional programming languages, making it easy to transfer design approaches and ideas from the functional world.

In this paper, we present the design and implementation of a GraphBLAS-like linear algebra foundation in two different settings: F# (a mature functional-first language, chosen for ease of prototyping and testing) and Inpla (a language and runtime for interaction nets). We employ quadtrees to represent sparse matrices and vectors, enabling natural divide-and-conquer parallelism through recursive subdivision. On this foundation, we build three canonical graph algorithms—breadth-first search (BFS), single-source shortest paths (SSSP), and triangle counting—using the algebraic primitives provided by our API.

Our contributions are as follows.

1. A type-driven design for a GraphBLAS-like API, demonstrating how to utilize algebraic types together with a minimal set of operations (e.g., `map`, `map2`, `spmv`, `spgemm`) for natural linear-algebra-based graph and graph analysis algorithms encoding with compile-time safety.
2. Reference implementations of a minimal, functionally equivalent subset of the API in F#, Vine, and Inpla, enabling direct comparison of respective execution models and runtime systems.
3. A comprehensive empirical evaluation of BFS, SSSP, and triangle counting across the three implementations, including:
    - runtime and memory comparison against C-based GraphBLAS baselines;
    - performance scaling analysis on multi-core architectures.

The paper is structured as follows. Section 2 provides background on GraphBLAS, quadtree matrix representations, interaction nets, and the Inpla language. Section **??** details our API design and its implementation in both F# and Inpla. Section 3.2 describes the graph algorithms built on this API. Section 4 presents our experimental setup and evaluation results. Section **??** discusses related work, and Section 5 concludes with directions for future research.

2. Background

In this section we briefly discuss GraphBLAS and introduce tree-based matrix representation. We also provide the basics of interaction nets, discuss implementations of interaction nets, and motivate our choice of which one we use for evaluation.

2.1. GraphBLAS

The goal of the GraphBLAS API is to provide basic building blocks for constructing high-performance graph analysis algorithms. To achieve this goal, GraphBLAS utilizes **sparse generic** linear algebra. Operations over sparse generic matrices and vectors parameterized by semiring-like structures provide a powerful basis for expressing a wide range of graph analysis algorithms [?].

GraphBLAS API: C API, C++ API, implementations in Java, Julia, etc.

Challenges on this path include the implementation of high-performance generic operations over sparse data structures. Imbalanced subtasks together with irregular data access patterns make load

balancing — and thus efficient parallel implementation — hard. Moreover, graph analysis algorithms are not single operations but nontrivial chains of operations over matrices and vectors. This leads to the problem of intermediate data structures. Kernel fusion could solve these problems, but it is too difficult to implement automatic optimizations for sparse generic computations, so it is done manually in the most frequent cases. Attempts to use MLIR (GraphBLAS dialect).!!! Poor utilization on GPUs. Nontrivial challenge to create generic kernels for GPU and FPGA.!!! There are also other problems discussed in "What Did GraphBLAS Get Wrong?" by John Gilbert[1]

SuiteSparse:GraphBLAS serves as the reference implementation of this API, offering a highly optimized, multi-threaded C library that supports various sparse matrix formats. Building on this foundation, LAGraph provides a collection of production-ready graph algorithms (e.g., PageRank, connected components, triangle counting) implemented using SuiteSparse:GraphBLAS. Currently it is one of the state-of-the-art (SOTA) solutions for high-performance graph analysis, thus we use it as a baseline for evaluation.

The long-term goal of our work is to investigate how interaction nets can help solve challenges with irregular parallelism and, possibly, with kernel fusion. In this work, we do not focus on the GraphBLAS API implementation, but we are inspired by it: we implemented a minimal set of generic data structures and operations sufficient to implement several classic graph analysis algorithms.

2.2. Tree-based Representation for Sparse Matrices and Vectors

To store sparse matrices and vectors efficiently, we employ a quad-tree data structure. The matrix is recursively subdivided into four quadrants — northwest (NW), northeast (NE), southwest (SW), and southeast (SE) — until each block becomes either uniform or empty. The Figure 1 shows an example of a matrix in the quad-tree form.

This recursive partitioning allows large uniform blocks to be represented implicitly, thereby reducing the memory required to store the matrix. Unlike flat array formats such as CSR or CSC, which are standard in libraries like SuiteSparse:GraphBLAS, the quad-tree naturally supports a divide-and-conquer execution model and adapts dynamically to irregular sparsity patterns. This structure simplifies parallelization of element-wise, matrix-matrix, and matrix-vector operations, as independent sub-blocks can be processed recursively without global synchronization.

However, this flexibility comes at a cost: for dense or highly regular matrices, the overhead of maintaining tree metadata and pointer indirection can outweigh the benefits of sparse storage, making traditional linear formats more efficient in such cases.

2.3. Interaction Nets

We utilize the variant of definitions provided by Y. Lafont in 1989 [?]. Interaction Nets is a graph-rewriting model where computation proceeds through local rewriting rules applied to pairs of connected nodes.

(Agent). An *agent* is a symbol from a fixed alphabet Σ, for which:

- an arity is defined — a function $Ar : \Sigma \to \mathbb{N} \cup \{0\}$;
- there is one *principal port* and a number of *auxiliary ports* equal to its arity.

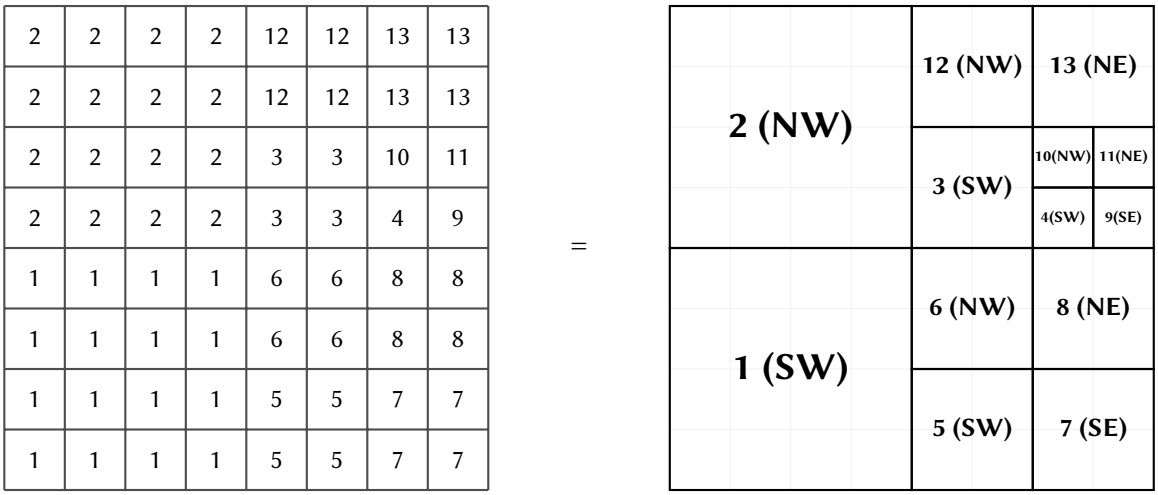
(Net). A *net* over the alphabet Σ is a labeled undirected multi-graph in which:

- each node is labeled by an agent;
- edges can only exist between ports;
- at most one edge enters each port.

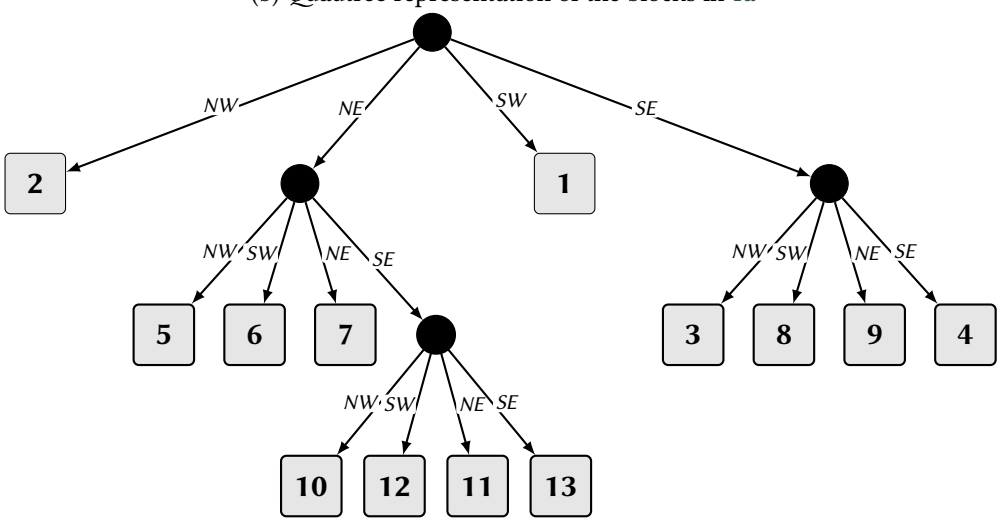
[1]Talk by John Gilbert, UC Santa Barbara, GraphBLAS BoF at HPEC, September 2022. Details can be found here: https://sites.cs.ucsb.edu/~gilbert/talks/talks.htm.

Figure 1: Example of the matrix and its quad-tree representation

(a) Block decomposition of the matrix



(b) Quadtree representation of the blocks in 1a



Let $\mathcal{N}_\Sigma$ denote the set of all possible nets over the alphabet Σ.

We say that a port is *free* if no edge originates from it. A net can also consist simply of a set of edges, in which case each end of an edge is considered a free port.

(Interface). The *interface* $I(n)$ is the set of all free ports of a net n.

We say that port p_a is connected to port p_b if there is an edge between them.

Computation in Interaction Nets occurs via *reductions*: rewriting an *active pair* into a net. An active pair is defined as two agents connected via their principal ports.

(Reduction Rule). A *reduction rule* is a graph rewriting rule. Let R be the set of reduction rules. More formally, a reduction rule is a triplet $r \in \Sigma \times \Sigma \times \mathcal{N}_\Sigma$, for which the following properties hold:

- **Symmetry:** $(l_1, l_2, n) \in R \Rightarrow (l_2, l_1, n) \in R$
- **Determinism:** $(l_1, l_2, n_1) \in R \wedge (m_1, m_2, n_2) \in R \Rightarrow (l_1, l_2) \neq (m_1, m_2)$
- **Interface Preservation:** $\forall (l_1, l_2, n) \in R, Ar(l_1) + Ar(l_2) = \|I(n)\|$

An example of the computation of the net encoding of the term $(0 + 1) + (0 + 1)$ is shown in Figure 2. The set of agents is $\Sigma = \{0, S, Add\}$ with arities $Ar(0) = 0$, $Ar(S) = 1$ and $Ar(Add) = 2$ and with the rules shown in Figure 3. The left auxiliary ports of the active Add-agents are connected to the net

representing 1, encoded in Peano numerals as $S(0)$, and the right auxiliary ports are connected to the third *Add*-agent. There is one free port — one of the auxiliary ports of the *Add*-agent. This constitutes the interface.

Figure 2: Example of computation in interaction nets: reduction of the term $(1 + 0) + (1 + 0)$ encoded in Peano numerals

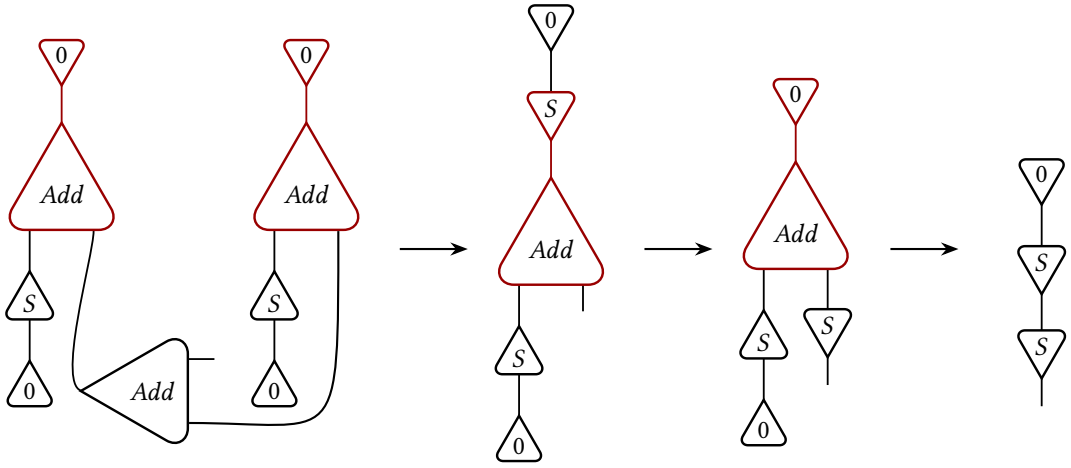
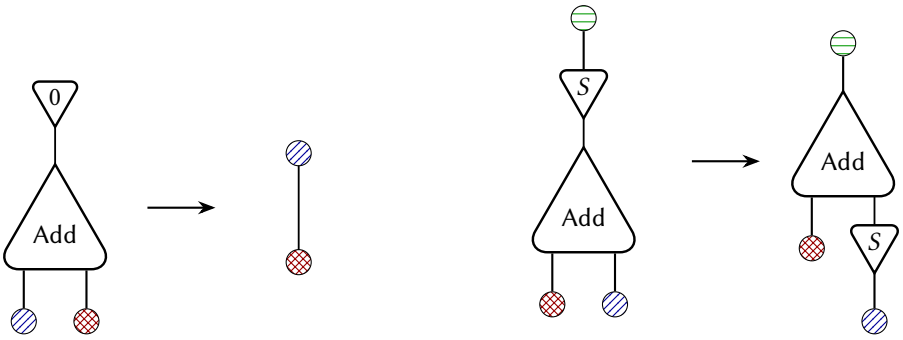


Figure 3: Reduction rules for Peano numerals



(a) Rule for $0 + x = x$

(b) Rule for $S(x) + y = S(x + y)$

As shown in Lafont's work [?], λ-calculus can be expressed through this formalism. Additionally, reductions can be performed independently of each other and it has been proven that the order of reductions doesn't matter. This serves as the primary reason for the natural suitability of Interaction Nets for parallel computations.

2.4. Implementations of Interaction Nets

Proposed by Lafont, interaction nets can serve as a foundation for irregular parallel computations.

Notable implementations include Inpla, HVM, and Vine. In OCaml [?] (Comparison, Inpla fastest)

Vine — high-level language (Rust-like) + interaction-nets-based intermediate representation + parallel virtual machine.

HVM — high-level language (JS-like) + interaction-nets-based intermediate representation + parallel virtual machine (+GPGPU?).

Inpla — mature enough, demonstrates good performance and scalability. Interaction-nets-based language + parallel VM.

No information is available on the evaluation or comparison of Vine and HVM. Typical benchmarks for these systems include Fibonacci, Ackermann, sorting, and list traversal. No graph analysis algorithms while such algorithms looks grate candidates for interaction nets. Other projects implement interaction nets in languages such as OCaml and Haskell.

We are focused on languages and runtimes.

In Haskell and OCaml

Inpla is a low-level interaction nets programming language, equipped with a parallel runtime. Programming in Inpla requires defining a custom set of rewriting rules for a chosen set of agents. Computation begins when two agents are connected via their primary ports.

```
// Defining custom rules
Add(r, x) >< Z =>
    r ~ x;
Add(r, x) >< S(y) =>
    Add(r, S(x)) ~ y;

// 1 + 1: connecting Add and S
Add(r, S(Z)) ~ S(Z);
// Printing the result and freeing the `r` name
r; free r;
```

Listing 1: Peano numerals addition in Inpla

Vine

IVY, IVM. Differences with Inpla: fixed set of agents (and respective rewriting rules).

HVM: fixed set of agents (and respective rewriting rules). Bend. Acceleration in Cuda. [2]

3. Implementation Details

In this section we describe our implementations: in F# (used as a prototype and reference for other implementations), in Inpla (a low-level language), and in Vine (a high-level language). We present the data structures and functions, list graph analysis algorithms (with short descriptions), and discuss details related to the implementation of the described structures and functions in selected languages.

3.1. Basic Data Structures and Functions

Matrix representation, as described in the F# code, is shown in Listing 2. Our representation is generic, meaning it is parameterized by the matrix cell type. The structure does not only work for matrices with sizes that are powers of two. A user-defined matrix of arbitrary size will be aligned to a square storage with size equal to a power of two using `Dummy` blocks. Since the representation is compressed, the `Storage` stores the real size (as a power of two) together with the respective tree. Finally, the `SparseMatrix` stores the matrix dimensions and the number of non-zero entries (nnz) together with data represented as `Storage`. Data structures for a sparse vector are described in a similar way.

We propose to use `Option<'T>` to explicitly represent cells that have no values. This approach is natural for graph representation: a cell of the adjacency matrix of a graph with edge labels of type `'T` has type `Option<'T>`: either there exists an edge with a label of the respective type (`Some(l)`), or there is no edge at all (`None`). Thus, for a sparse matrix with filled cells of type `'T`, the storage has type `Option<'T>`.

We implemented a minimal set of operations over matrices and vectors required to implement selected graph analysis algorithms. The list of basic functions includes `map`, `fold`, `reduce`. Element-wise operations are based on `map2`. Particularly, `mask` is implemented as a case of `map2`. Linear algebra functions include: `vxm` (generic vector-matrix multiplication) and `mxm` (generic matrix-matrix multiplication). We also support transpose and extracting the triangular part of a matrix (required

[2] Issues, problems, eg with QTree

```
type treeValue<'value> = | Dummy | UserValue of 'value

(*
| x1 | x2 |
----------
| x3 | x4 |
*)
type qtree<'value> =
    | Node of qtree<'value> * qtree<'value> * qtree<'value> * qtree<'value>
    | Leaf of treeValue<'value>

[<Struct>]
type Storage<'value> =
    // Storage is always size-x-size square.
    val size: uint64<storageSize>
    val data: qtree<'value>
    new(_size, _data) = { size = _size; data = _data }

[<Struct>]
type SparseMatrix<'value> =
    val nrows: uint64<nrows>
    val ncols: uint64<ncols>
    val nvals: uint64<nvals>
    val storage: Storage<Option<'value>>
```

Listing 2: Matrix representation

for the triangle counting algorithm). We did not implement fused versions. Conversion to and from coordinate list format is provided to simplify input graph building. All operations take sparsity into account and preserve compression. No manual parallelization to !!!

Exception handling is unlikely to be possible in interaction nets in the traditional way due to their highly parallel and irregular nature. Moreover, the selected languages (Inpla and Vine) do not provide any specific mechanisms for exception handling. We focus on realistic code, so we should handle not only correct cases but also erroneous ones (e.g., inconsistent matrix or vector sizes). For this, we use the `Result` type (even in the F# version)[3] that explicitly represents two cases: `Ok<'R>` for successfully completed computations and `Err<'E>` for errors. This way of handling errors is represented in many languages such as Rust, F#, etc.

3.2. Implemented Graph Analysis Algorithms

We implemented three basic graph analysis algorithms.

- Level-based BFS: for the given graph and start vertex, it returns a sparse vector `result` such that if `result[i] = Some(k)`, then vertex `i` was visited at step `k`. If `result[i] = None`, then `i` is unreachable.
- SSSP: for the given graph and start vertex, it returns a sparse vector `result` such that if `result[i] = Some(k)`, then the shortest path from the start vertex to `i` has length `k`. If `result[i] = None`, then `i` is unreachable. To prevent an infinite loop in the case of negative cycles, we check that the number of iterations should not be greater than the number of vertices in the input graph.

[3]We do this to be as close to interaction-net-based implementations as possible. The F# language has traditional exception mechanisms.

- Triangle counting algorithm.

All algorithms were implemented using operations over matrices and vectors similarly to how they are described in GraphBLAS-related materials. We provide basic implementations without any optimizations such as the *push-pull* optimization for BFS or SSSP.

3.3. Interaction Nets Related Details

The translation is almost "as-is" without any application of interaction-net-specific optimizations or code templates.

In general, a high-level programming language function is translated into an interaction nets agent with a single port and an associated rewriting rule. Calling such a function involves connecting the function agent to its argument (which may be represented as a tuple agent). The result is then obtained on the agent's port after a sequence of rewritings. This defines an interface function, which is typically rewritten into a single implementation agent with more complex interaction rules.

```
// Simple interaction rule; real implementation omitted
F(r) >< (arg1, arg2, arg3) =>
    f_implementation(r, arg1, arg2) ~ arg3;

// Calling the interface function with real data
F(r) ~ (A(x, y), B, Some(1));
r; free r;
```

Listing 3: Interface function pattern in Inpla

Inpla has no I/O operators, so input data structures must be generated in advance. Error handling is minimal and relies on the virtual machine's "no interaction rule" exception for unforeseen situations, as implementing comprehensive checks would introduce significant cognitive overhead in such a low-level language.

A significant concern is memory consumption. Interaction nets programming requires the use of duplicator agents for data reuse. Inpla's current implementation eagerly copies entire subnets when duplicating, resulting in substantial memory overhead compared to the F# prototype. Addressing this issue requires further investigation.

Vine supports reading. Execution is parallel, so separating concerns is difficult. Error handling uses the standard `Result` type from the Vine language.

Inpla does not support floats. Vine supports floats but without comparison (possibly due to problems with NaN comparison). Therefore, weights in SSSP are represented as integers, not floats, in both Vine and Inpla implementations.

4. Evaluation

Our goal to estimate ... To detect problems....

4.1. Experimental Setup

Hardware. All experiments were conducted on a system with an AMD Ryzen 7 5800X 8-core processor (16 threads with hyper-threading enabled, base clock 3.8 GHz) and 128 GiB of DDR5 RAM.

Software. We compare four implementations:

- **F#**: Our quadtree-based implementation, compiled with F# !!! and the .NET !!! runtime. This serves as a baseline representing a conventional functional language approach. Single thread. Benchmarkdotnet for benchmarking automation.
- **Inpla**: Our interaction-net-based implementation, executed on the Inpla runtime system (version 0.13.4, patched). Settings: a 120 GB heap using a patched version of Inpla, allowing arbitrary heap size configuration with per-number-of-threads adjustments via scripting. The patch also increases several virtual machine compile-time constants to support very large trees.
  A fatal bottleneck in list parsing was discovered during testing, and the fix has since been merged into the main Inpla repository.
- **Vine**: Our interaction-net-based implementation, executed on the Vine runtime system (version !!!). Settings: 110Gb heap per worker.
- **LAGraph**: A collection of graph algorithms built on SuiteSparse:GraphBLAS (version !!!), providing state-of-the-art, linear-algebra-based reference implementations of BFS, SSSP, and triangle counting.
- **NetworkX**: A popular Python library for graph analysis (version !!!), included as a baseline representing ease of use over optimization.

Datasets. We evaluate our implementations on several real-world and synthetic graphs from the SuiteSparse Matrix Collection [**?**]. The graphs are chosen to represent a variety of sizes, sparsity patterns, and application domains. Table 1 summarizes their key properties: number of vertices ($|V|$), number of edges ($|E|$), and degree statistics (average, maximum, and median).

Table 1
Dataset properties

Graph	$\|V\|$	$\|E\|$
coAuthorsCiteseer	227,320	1,628,268
boneS01	127,224	5,516,602
usroads-48	126,146	323,900
Andrews	60,000	760,154
Trefethen_2000	2000	41,906
gr_30_30	900	7,744

Patterns.

Research Questions. Our evaluation addresses the following research questions:

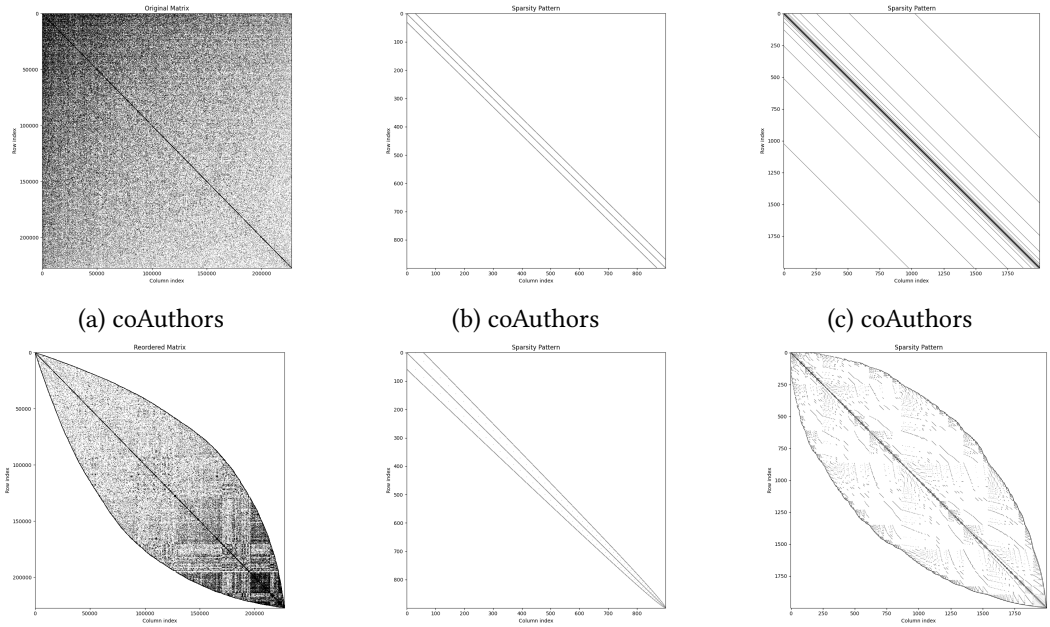
RQ1 (Correctness): Do the F# and Inpla implementations produce correct results for BFS and triangle counting on all datasets, when compared against reference outputs from NetworkX and LAGraph?

RQ2 (Performance – Inpla vs. Vine vs. State-of-the-Art): How does the execution time of Inpla compare to LAGraph on BFS, SSSP, and triangle counting? We also include F# and NetworkX as additional baselines representing different points in the performance–productivity trade-off.

RQ3 (Parallel Scaling): How well do the Inpla, Vine, and LAGraph implementations scale with the number of available CPU cores? We measure speedup from 1 to all available cores for BFS, SSSP, and triangle counting, with hyper-threading both enabled and disabled.

RQ4 (Impact of Matrix Reordering): Matrix reordering techniques (e.g., reverse Cuthill–McKee) can improve locality and cache performance.

Even for [**?**] How does applying such reordering affect the performance of Inpla, F#, and LAGraph implementations?



(a) coAuthors (b) coAuthors (c) coAuthors

(d) coAuthors reordered (e) coAuthors reordered (f) coAuthors reordered

Figure 4: Matrices shapes: original and after reordering

4.2. Evaluation Results

Graphics, tables.

Reordering: figures with sparsity patterns

Analysis.

Technical. Instability of Inpla. Data races. memory management.

5. Conclusion and Future Work

We have demonstrated a feasible mapping of GraphBLAS concepts to interaction nets using quadtrees, provided concrete implementations in Inpla and Vine, and evaluated their scalability. The performance gap compared to state-of-the-art parallel linear-algebra-based frameworks remains substantial, and scalability is limited.

Languages: But for comfortable usage `Result` monad or similar mechanisms is required.

Several bottlenecks and limitations were identified. The naive implementation suffers from a duplication problem: a huge number of duplicated nodes limits both performance and memory efficiency. Reordering (RCM) is crucial for performance and memory consumption but is not handled automatically. Currently it is a separated preprocessing step. Moreover, only a limited set of operations and algorithms has been evaluated.

Future work includes investigating improved duplication strategies. The set of implemented operations and algorithms should be extended to include indexed operations, slices, and such graph analysis algorithms as PageRank, minimum spanning forest (MSF), regular path queries, context-free path queries (CFPQ), and maximal independent set (MIS).

A systematic evaluation of other interaction-net-based implementations, such as HVM, is also required. Alternative sparse matrix representations, such as the coordinate list (COO) format, rely on sorting—an operation that parallelizes well in interaction nets and therefore warrants further investigation.

Classical implementations of algorithms. May interaction nets be good enough to parallel straightforward implementations?

Automatic kernel fusion presents another direction. Advanced optimization techniques natural to functional programming, such as supercompilation [?] or distillation [?], could be applied to

automatically fuse sequences of operations, analogous to manual fusion for masks and other constructs in classic GraphBLAS. Partial evaluation for interaction nets [?] Whether these techniques can be adapted for interaction nets remains an open question.

Finally, hardware acceleration remains an open opportunity. Designing an FPGA specifically optimized for interaction net reduction could exploit the fine-grained parallelism inherent to the model and yield substantial performance gains.

6. Code Availability Statement

All implementations, resources and scripts for evaluation available at GitHub.

- F# prototyping code: https://github.com/Lamagraph/QTreeFSharp.
- Inpla implementation: https://github.com/Lamagraph/QTreeInpla.
- Vine implementation: https://github.com/Lamagraph/QTreeVine.
- Patched version of Inpla: https://github.com/Lamagraph/inpla
- Benchmarking scripts and dataset configurations.

Acknowledgments

Thanks to the developers of ACM consolidated LaTeX styles https://github.com/borisveytsman/acmart and to the developers of Elsevier updated LaTeX templates https://www.ctan.org/tex-archive/macros/latex/contrib/els-cas-templates.

Declaration on Generative AI

During the preparation of this work, the author(s) used X-GPT-4 and Gramby in order to: Grammar and spelling check. Further, the author(s) used X-AI-IMG for figures 3 and 4 in order to: Generate images. After using these tool(s)/service(s), the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication's content.